Algoritmos e Estruturas de Dados II

Prof. Henrique Batista da Silva

Recursividade

Recursividade

- Uma definição recursiva consiste em duas partes:
  - Caso base: elementos básicos
  - Regras de construção de novos objetos a partir dos elementos básicos.
- Exemplo (construção do conjunto de números naturais) :
  - $0 \in \mathrm{N}$;
  - Se $n \in \mathrm{N}$, então $(n + 1) \in \mathrm{N}$;

Recursividade

- Um procedimento recursivo caracteriza-se por chamar a si mesmo.
- Recursividade é útil para descrever algoritmos mais claros e concisos.
- Exemplo de função recursiva:
  - Função que define elevar qualquer número x a uma potência inteira não negativa n.
  - Notação:

$$x^n = \begin{cases} 1 & se\ n = 0 \\ x\ .\ x^{n-1} & se\ n > 0 \end{cases}$$

Caso base

Passo de indução

Recursividade

- Por exemplo: função fatorial (!).

$$n! = \begin{cases} 1 & se\ n = 0 \quad \text{(caso base)} \\ n\ .\ (n-1)! & se\ n > 0 \quad \text{(passo de indução)} \end{cases}$$

- Para o primeiro caso, a solução foi dada por definição, não necessita da recursão para ser alcançada.
- Para o segundo caso, pode ser resolvido parte do problema, menor e mais simples.
- Por exemplo, sabemos que *0!* = 1, e sabemos que *1! = 1 x 0!,* portanto *n! = n x (n – 1)!*

Recursividade

- As definições recursivas têm um efeito indesejado, pois é necessário calcular todos os números da sequência para descobrir o fatorial de *n!*

- No entanto, encontrar uma fórmula para solucionar o problema sem referenciar outros elementos da sequência é um problema difícil e nem sempre pode ser resolvido.

Chamada de função

- Uma função pode ser chamada por outras funções ou pelo programa principal (função *main*).

- A indicação de onde a função foi chamada deve ser armazenada pelo sistema.

- Nas chamadas de funções recursivas uma pilha é utilizada para armazenar o estado da cada função, quando esta foi chamada.

Chamada de função

- O estado da função é caracterizado pelo conteúdo de todas as **variáveis**, seus **parâmetros** e **endereço de retorno**.

- O endereço de retorno serve para que o sistema “lembre” onde deve retornar a execução depois que a função terminar.

Chamada de função

- Sempre que uma função é iniciada recursivamente, um novo conjunto de variáveis locais e de parâmetros é alocado.

- Quando ocorre um retorno, este conjunto é desativado, dando lugar ao conjunto de valores da função que fez a chamada.

Chamada recursiva

- Todo algoritmo deve terminar após a execução de uma quantidade finita de passos.

- Uma função recursiva deve parar em algum instante, para isto ela deve atingir, em algum momento, o **caso base**, ou seja, sua condição de parada.

- Caso contrário, a função poderá entrar em *looping* e ser executado infinitamente → estouro de pilha do sistema.

Chamada recursiva

- No exemplo do cálculo da potência... :

$$x^n = \begin{cases} 1 & se\ n = 0 \quad \text{(caso base)} \\ x\ .\ x^{n-1} & se\ n > 0 \quad \text{(passo de indução)} \end{cases}$$

- ... a aplicação repetitiva da etapa indutiva leva ao caso base (condição de parada). Última etapa na cadeia de chamadas recursivas.

Cálculo da Potência (Recursivo)

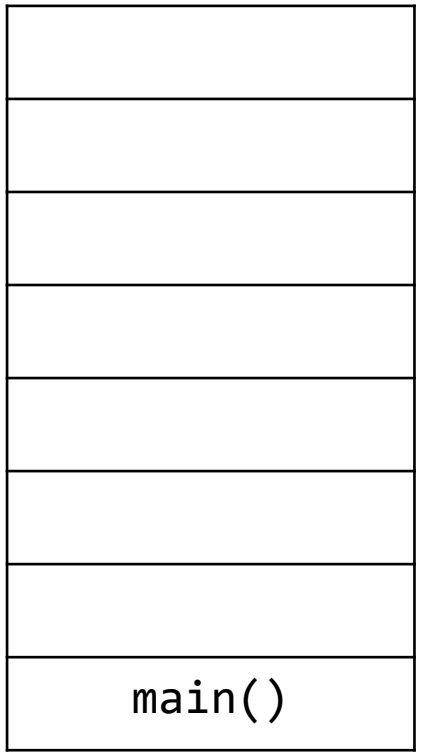
- Função que calcula x^n pode ser escrita diretamente a partir da definição de uma potência:

```
int potencia(int x, int n)  {
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}
```

Cálculo da Potência (Recursivo)

```c
int potencia(int x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}

int main()
{
    int x = 2;
    int n = 3;

    int pot = potencia(x, n);

    printf("Resultado: %d", pot);

    return 0;
}
```

Pilha de execução do programa



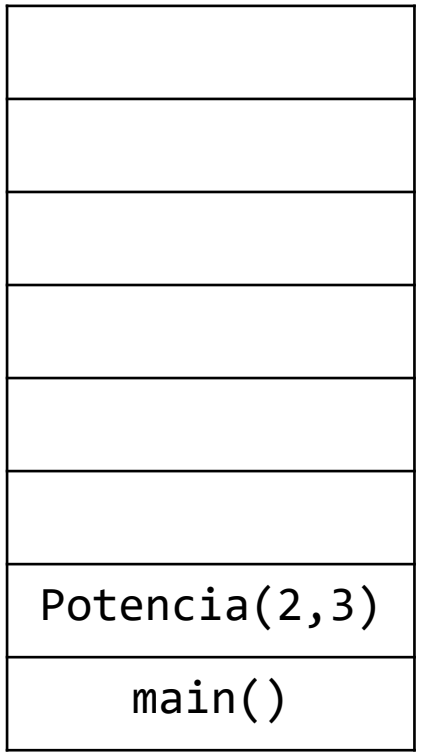
Ao iniciar um programa, a chamada da função principal é armazenada na pilha de execução do programa. Esta pilha armazena os valores das variáveis (posições de memória) do programa naquele momento, seus parâmetros e endereço de retorno.

O endereço de retorno serve para que o sistema saiba onde deve retornar a execução depois que a função terminar.

Cálculo da Potência (Recursivo)

```
int potencia(int x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}

int main()
{
    int x = 2;
    int n = 3;

    int pot = potencia(x, n);

    printf("Resultado: %d", pot);

    return 0;
}
```

Pilha de execução do programa



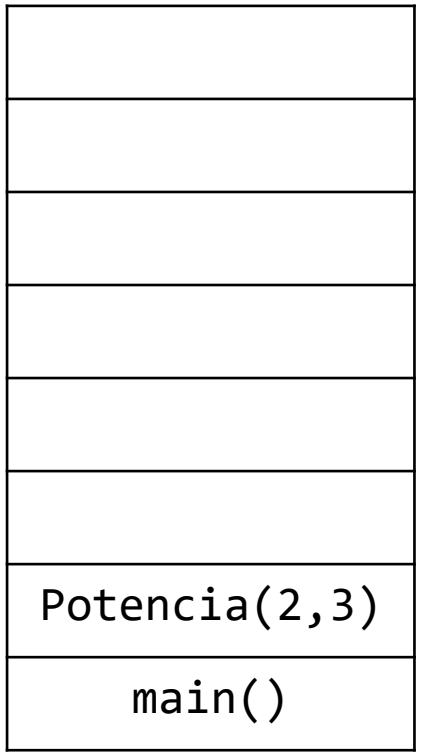
Sempre que uma função chama outra função, todas as suas informações (como os valores de suas variáveis) são armazenada na pilha de execução.

Cálculo da Potência (Recursivo)

```
int potencia(int x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}

int main()
{
    int x = 2;
    int n = 3;

    int pot = potencia(x, n);

    printf("Resultado: %d", pot);

    return 0;
}
```

Após a verificação da condição **if, c**omo n é diferente de zero, chama a função novamente, mas agora decrementando o valor de n (observe que não ocorre o retorno ainda)

Pilha de execução do programa

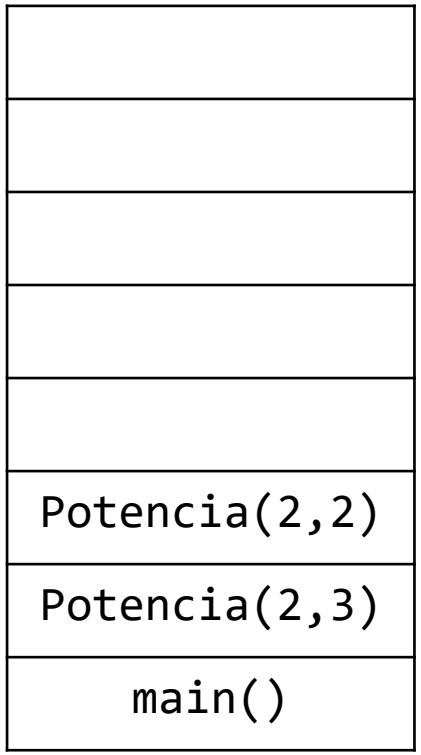


Cálculo da Potência (Recursivo)

```
int potencia(int x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}

int main()
{
    int x = 2;
    int n = 3;

    int pot = potencia(x, n);

    printf("Resultado: %d", pot);

    return 0;
}
```

Nova chamada da função. Observe os valores do parâmetros que são passados para a nova chamada.

Pilha de execução do programa

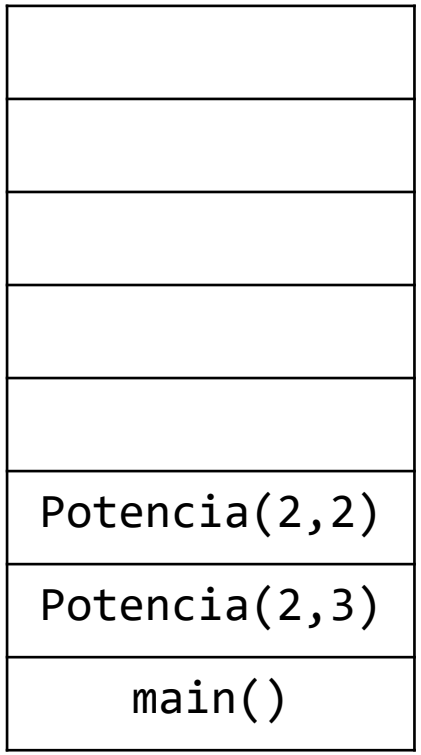


Cálculo da Potência (Recursivo)

```
int potencia(int x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}

int main()
{
    int x = 2;
    int n = 3;

    int pot = potencia(x, n);

    printf("Resultado: %d", pot);

    return 0;
}
```

Como n ainda é diferente de zero, chama a função novamente, mas agora decrementando o valor de n

Pilha de execução do programa

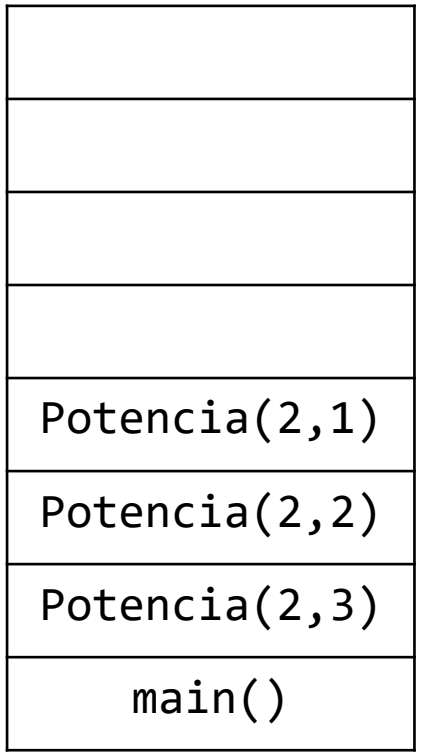


Cálculo da Potência (Recursivo)

```
int potencia(int x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}

int main()
{
    int x = 2;
    int n = 3;

    int pot = potencia(x, n);

    printf("Resultado: %d", pot);

    return 0;
}
```

Nova chamada da função. Observe os valores do parâmetros que são passados para a nova chamada.

Pilha de execução do programa

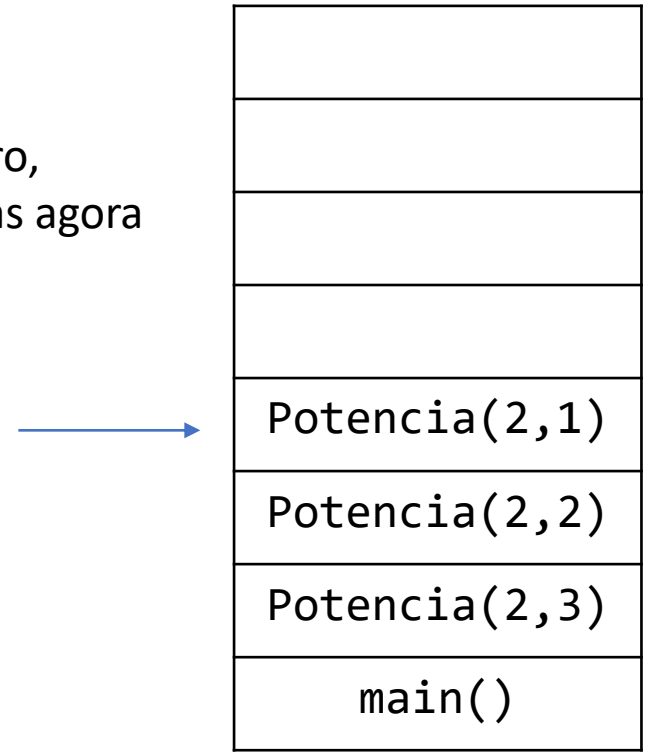


Cálculo da Potência (Recursivo)

```
int potencia(int x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}

int main()
{
    int x = 2;
    int n = 3;

    int pot = potencia(x, n);

    printf("Resultado: %d", pot);

    return 0;
}
```

Como n ainda é diferente de zero, chama a função novamente, mas agora decrementando o valor de n

Pilha de execução do programa

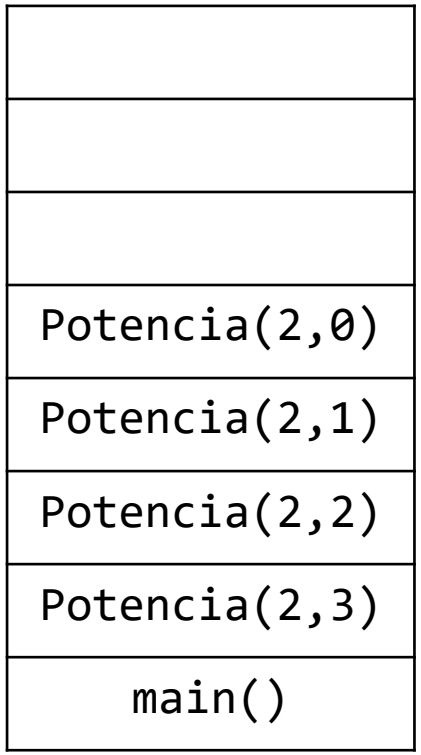


Cálculo da Potência (Recursivo)

```
int potencia(int x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}

int main()
{
    int x = 2;
    int n = 3;

    int pot = potencia(x, n);

    printf("Resultado: %d", pot);

    return 0;
}
```

Nova chamada da função. Observe os valores do parâmetros que são passados para a nova chamada.

Pilha de execução do programa

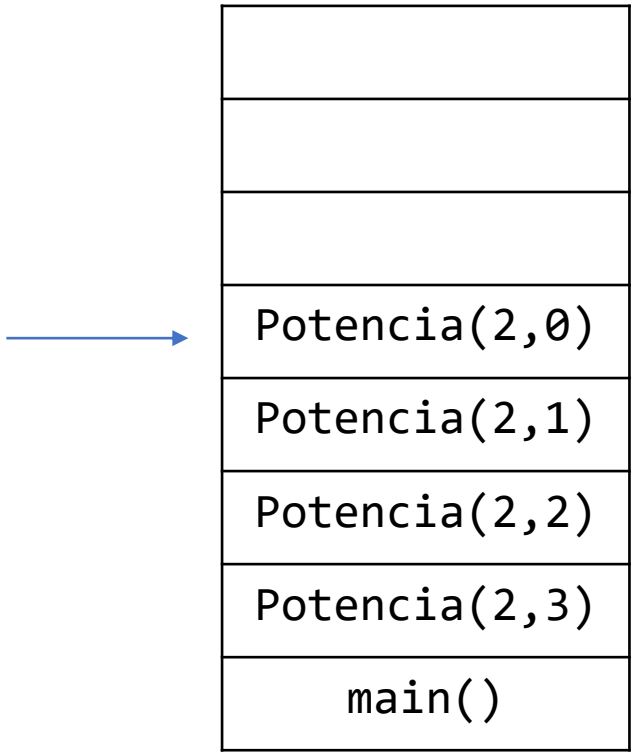


Cálculo da Potência (Recursivo)

```c
int potencia(int x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}

int main()
{
    int x = 2;
    int n = 3;

    int pot = potencia(x, n);

    printf("Resultado: %d", pot);

    return 0;
}
```

Como n é igual a zero, apenas retorna 1 para a chamada imediatamente anterior da própria função.
As chamadas recursivas acontecem até que o caso base seja alcançado (condição de parada).

Pilha de execução do programa

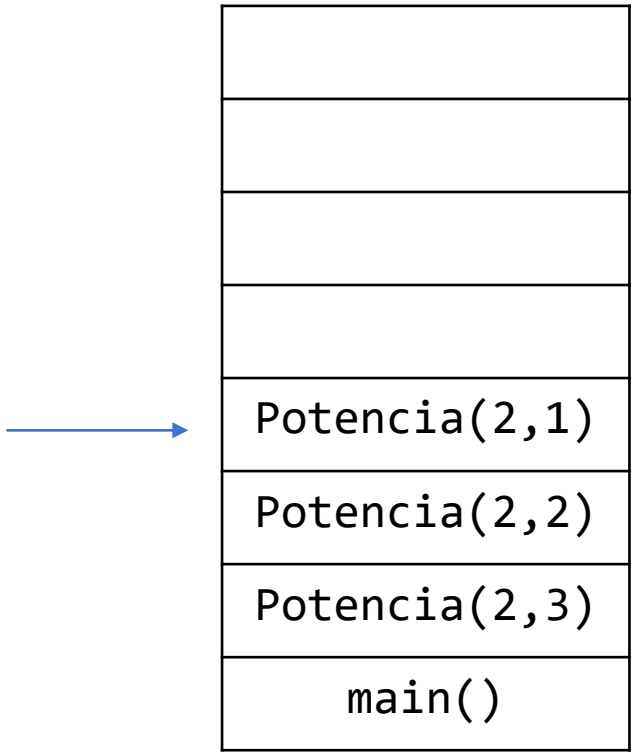


Cálculo da Potência (Recursivo)

```
int potencia(int x, int n)
{
    if (n == 0)
        return 1;
    else                        = 1
        return x * potencia(x, n - 1);
}

int main()
{
    int x = 2;
    int n = 3;

    int pot = potencia(x, n);

    printf("Resultado: %d", pot);

    return 0;
}
```

Valor de retorno da chamada anterior (1). Esta chamada atual irá retornar o valor da chamada anterior (igual a 1) vezes “x” (sempre igual a 2).

Pilha de execução do programa

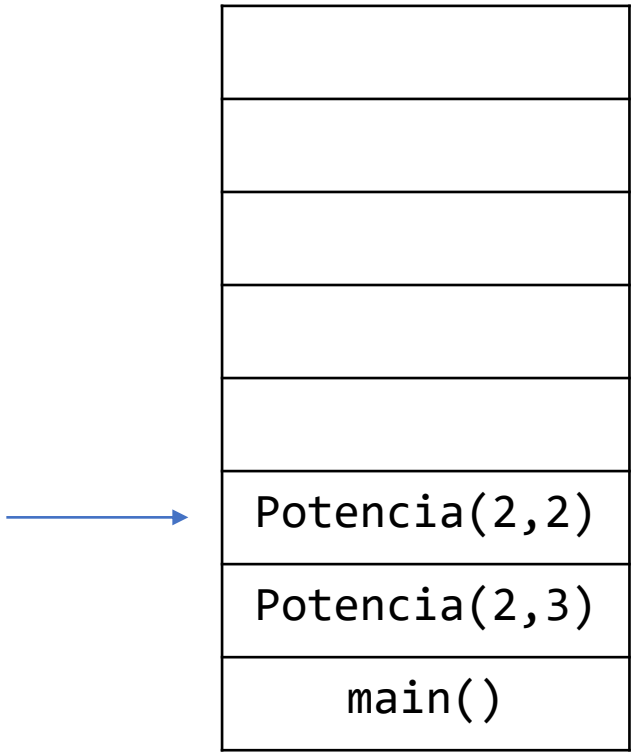


Cálculo da Potência (Recursivo)

```
int potencia(int x, int n)
{
    if (n == 0)
        return 1;
    else                    = 2
        return x * potencia(x, n - 1);
}

int main()
{
    int x = 2;
    int n = 3;

    int pot = potencia(x, n);

    printf("Resultado: %d", pot);

    return 0;
}
```

Valor de retorno da chamada anterior (2 * 1). Esta chamada atual irá retornar o valor da chamada anterior (igual a 2) vezes “x” (sempre igual a 2).

Pilha de execução do programa

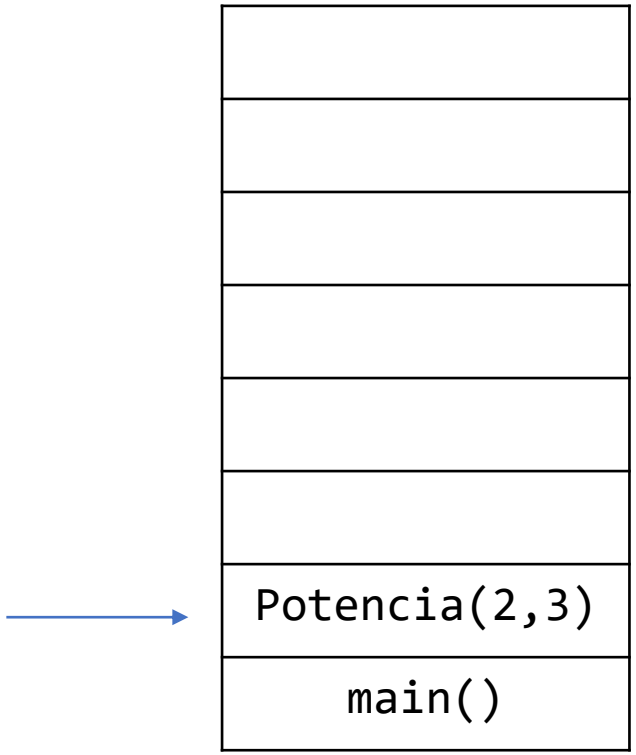


Cálculo da Potência (Recursivo)

```
int potencia(int x, int n)
{
    if (n == 0)
        return 1;
    else                    = 4
        return x * potencia(x, n - 1);
}

int main()
{
    int x = 2;
    int n = 3;

    int pot = potencia(x, n);

    printf("Resultado: %d", pot);

    return 0;
}
```

Valor de retorno da chamada anterior (2 * 2). Esta chamada atual irá retornar o valor da chamada anterior (igual a 4) vezes “x” (sempre igual a 2).

Pilha de execução do programa



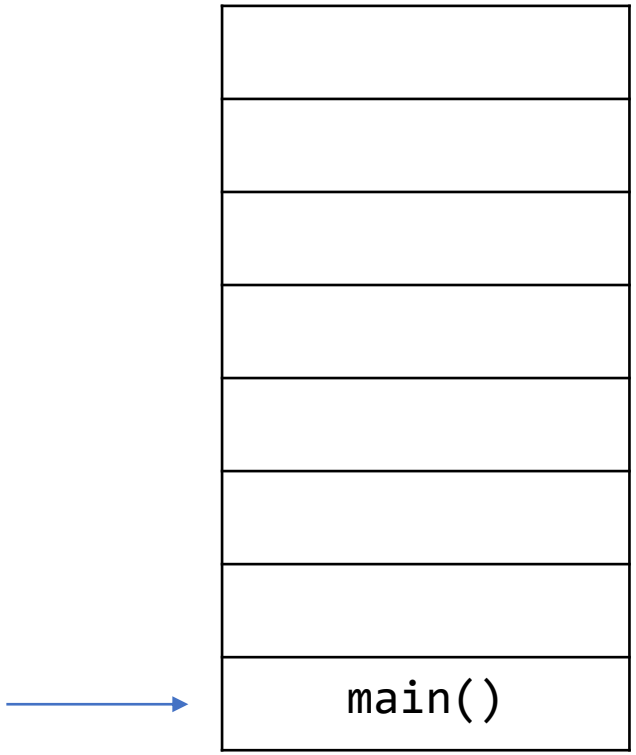
Cálculo da Potência (Recursivo)

```
int potencia(int x, int n)
{
    if (n == 0)
        return 1;
    else
        return x * potencia(x, n - 1);
}

int main()
{
    int x = 2;
    int n = 3;
                        = 8
    int pot = potencia(x, n);

    printf("Resultado: %d", pot);

    return 0;
}
```

Valor de retorno da primeira chamada da função potencia() pela função main(). A variável “pot” receberá o valor 8 (resultado de 2 elevado a 3).

O valor 8 será exibido na tela e o programa será finalizado.

Pilha de execução do programa



Cálculo de Fibonacci (Recursivo)

- Encontrar o enésimo termo da sequência de Fibonacci.
- Notação:

$$fib(n) = \begin{cases} 0 & se\ n = 1 \\ 1 & se\ n = 2 \\ fib(n-1) + fib(n\ -2) & se\ n > 2 \end{cases}$$

Cálculo de Fibonacci (Recursivo)

- Encontrar o enésimo termo da sequência de Fibonacci.

```
int fibonacci(int n)
{
    if (n == 1)
        return 0;
    else if (n == 2)
        return 1;
    else
        return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {

    int n = fibonacci(10);
    printf("Resultado: %d", n);

    return 0;
}
```

Recursividade: Exercícios

- Faça uma função que calcule o fatorial de N, recebido como parâmetro.

- Faça uma função que recebe como parâmetro um inteiro positivo *N* e retorne o somatório de todos os números inteiros entre 0 e *N*.